

Capitolul 8. Arhitectura microserviciilor folosind Docker și Kubernetes

8.1. Principiile decuplării serviciilor Dezvoltarea sistemelor informatice distribuite a impus o schimbare de paradigmă de la arhitecturile monolitice către modelele bazate pe microservicii. Această abordare presupune împărțirea unei platforme complexe într-o suită de procese mici, independente, care comunică între ele prin interfețe bine definite, de regulă API-uri de tip REST sau gRPC. Fiecare microserviciu este responsabil de o funcționalitate distinctă a afacerii și poate fi dezvoltat, testat și lansat independent, folosind stiva tehnologică optimă pentru problema pe care o rezolvă. Această independență facilitează o scalabilitate granulară, permițând alocarea resurselor hardware strict acolo unde există blocaje de performanță.

8.2. Containerizarea aplicațiilor cu Docker Pentru a asigura un mediu de execuție consistent între etapa de dezvoltare și cea de producție, tehnologia containerelor a devenit un standard în industrie. Docker permite împachetarea unui microserviciu împreună cu toate dependențele sale, bibliotecile și fișierele de configurare într-o singură unitate standardizată numită imagine. Containerele rulează direct pe nucleul sistemului de operare gazdă, fiind mult mai ușoare și mai rapide decât mașinile virtuale tradiționale. Astfel, se elimină complet problema clasică a incompatibilităților de mediu, asigurând o portabilitate maximă a codului pe orice infrastructură de procesare.

8.3. Orchestrarea la scară largă prin Kubernetes Pe măsură ce numărul de containere dintr-un ecosistem crește, gestionarea manuală a ciclului lor de viață devine imposibilă. Kubernetes a apărut ca o soluție de orchestrare absolut necesară pentru automatizarea lansării, scalării și operării aplicațiilor containerizate. Prin intermediul podurilor și al nodurilor de calcul, Kubernetes monitorizează constant starea sistemului. Dacă un container eșuează din cauza unei erori interne, planificatorul îl repornește automat pe un alt nod disponibil, asigurând o disponibilitate ridicată (High Availability). De asemenea, distribuitorul de trafic integrat direcționează solicitările către instanțele active, optimizând astfel consumul de resurse la nivel de cluster.